CS 4803/8803

P3c: Reflection Questions

Ojas Mediratta
omediratta3@gatech.edu
903678112

Provide your answer on the appropriate page. Include screenshots where requested.
You may add new pages but ensure you assign them to the appropriate question on Gradescope.
Submit as a PDF to Gradescope “Project 3C - Reflection”

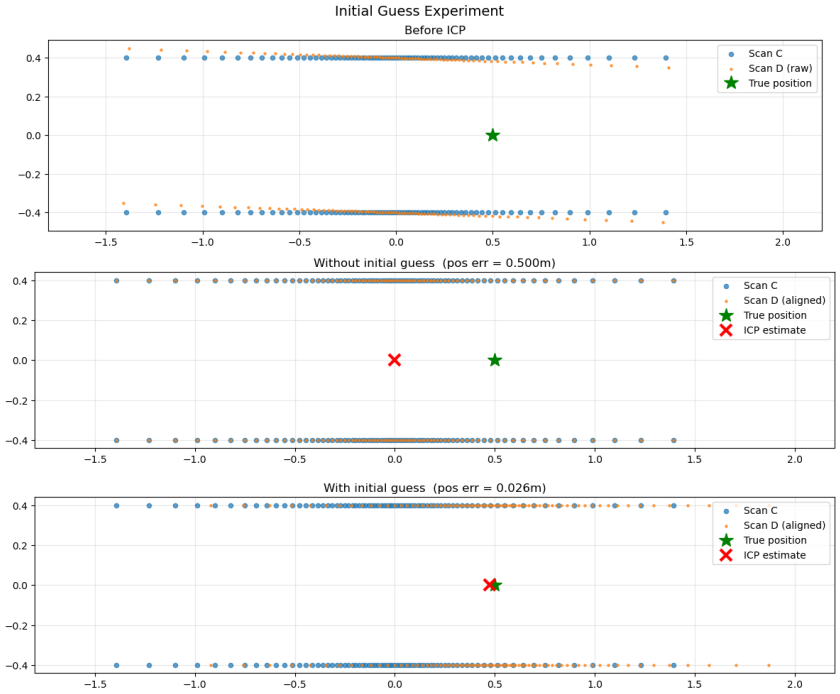
Q1. In the corridor experiment, ICP without an initial guess reports low correspondence error but has ~0.5m position error.

(a) Explain why this happens.

(b) What does this tell you about using ICP error alone to judge alignment quality?

(c) In your SLAM pipeline, where does the initial guess for ICP come from, and why does it help?

Include a screenshot of the corridor ICP results (with and without initial guess). [4 pts]



a) The initial guess matters a lot because ICP is a local optimization method. It does not search for the best alignment globally, so it usually converges to whatever nearby solution looks best from its starting point.

b) Without an initial guess, ICP converges to the wrong alignment because this scene is ambiguous. The two long parallel walls look very similar across many horizontal shifts, so ICP can slide the scan along the walls and still get a low error. That is why the estimate ends up far from the true pose, with about 0.500 m error.

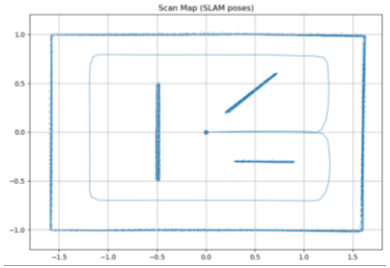
c) In the SLAM pipeline, the initial guess for ICP comes from odometry. Then it converges to the correct local minimum, giving a much smaller error of about 0.026 m. This shows why ICP is usually combined with odometry: odometry gives a coarse estimate, and ICP refines it.

Q2. Run your single-robot SLAM in three configurations by changing only odom_sigmas:

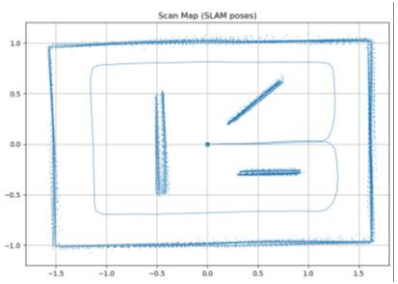
(a) Default: [0.005, 0.005, 0.002] (b) 100x higher: [0.5, 0.5, 0.2] (c) Very large: [100, 100, 100]

Compare the resulting scan maps and RMSE. What is the optimizer effectively doing in configuration (c)? Is (b) closer to (a) or (c)? Why? What does this tell you about the role of covariance in sensor fusion?

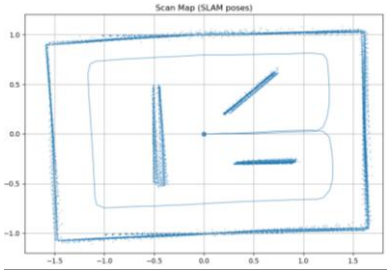
Include screenshots of all three scan maps and report the RMSE for each. [4 pts]



[0.005, 0.005, 0.002]
slam error 0.7 cm



[100, 100, 100]
slam error 2.0 cm



[0.05, 0.05, 0.02]
slam error 2.5 cm

In configuration (c), the optimizer is basically ignoring odometry and relying mostly on the ICP factors plus the prior. That is why the map becomes more warped and the RMSE gets worse.

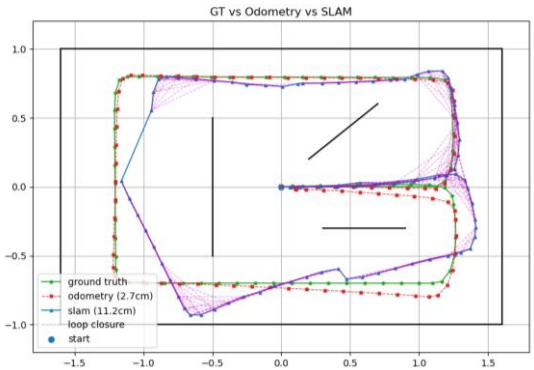
Configuration (b) is closer to (c) than to (a), because increasing the odometry sigmas by 100x already makes odometry much less trusted. So the solution starts behaving more like scan-matching-dominated SLAM than the default balanced case.

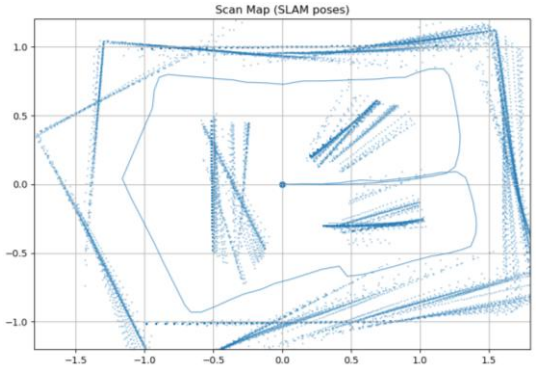
This shows that covariance controls how much the optimizer trusts each sensor. In sensor fusion, the result depends not just on the measurements, but on their assigned uncertainty.

Q3. Pick one of the following parameters and describe what happens when you change it significantly:

(a) search_radius (b) icp_threshold (c) KEYFRAME_EVERY

Report the RMSE and loop closure count for your original and modified values. Why does this parameter matter? Include screenshots of the scan maps. [4 pts]





I changed `icp_threshold` and compared the original and modified results. With the original value, the odometry RMSE was 2.7 cm, the SLAM RMSE was 2.5 cm, and the loop closure count was 1492. After changing the parameter, the odometry RMSE stayed 2.7 cm, but the SLAM RMSE increased to 11.2 cm and the loop closure count increased to 1633.

This shows that `icp_threshold` matters because it controls how strict the algorithm is when accepting ICP matches as loop closures. With the looser setting, more loop closures were added, but some were likely poor matches, which distorted the graph and made the SLAM result worse.

So in this case, the original threshold worked better because it accepted fewer but more reliable loop closures. This gave a much lower SLAM RMSE and a cleaner map.

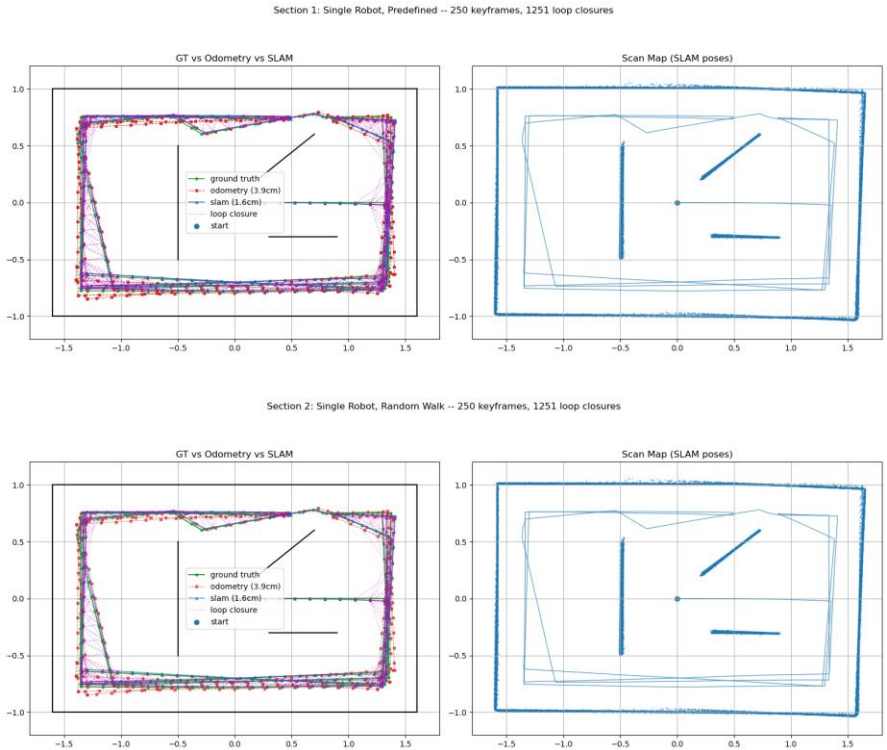
Q4. Compare your scan maps from Section 1 (predefined trajectory) and Section 2 (random walk).

(a) Is the map quality different? Which is crisper?

(b) How many loop closures did each produce?

(c) Why might a predefined trajectory produce a cleaner (or worse) map than a random walk?

Include screenshots of both scan maps. [4 pts]



a) The map quality was not very different in my results. Both scan maps looked fairly crisp and captured the same overall structure of the environment. If anything, the predefined trajectory looked only slightly cleaner, but the difference was small.

b) The predefined trajectory produced 1251 loop closures which was the same as the random walk.

c) A predefined trajectory can sometimes produce a cleaner map because it revisits the environment in a more regular way, which can lead to more consistent loop closures. A random walk is less structured, so it may produce fewer revisits or less consistent scan alignments. However, in my case both maps were very similar, so this suggests the SLAM pipeline was robust enough that both trajectories gave good coverage and enough useful constraints.

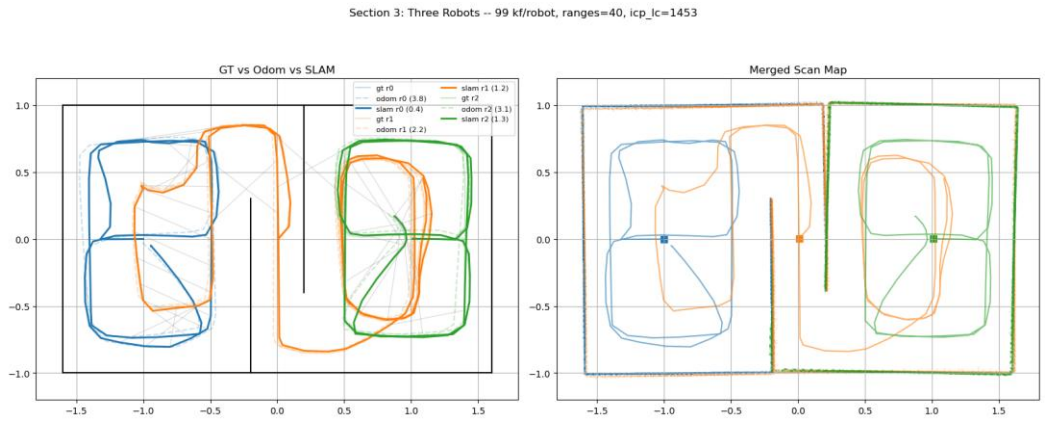
Q5. In Section 3, robot 0 (left) and robot 2 (right) never come within range of each other.

(a) How does the optimizer place them in the same global coordinate frame?

(b) What role does robot 1 play?

(c) What would happen to the map if robot 1 were removed entirely?

Include a screenshot of your Section 3 merged scan map. [4 pts]



a) The optimizer places robots 0 and 2 in the same global coordinate frame through the connected factor graph. Even though robots 0 and 2 never directly come within range of each other, they are both connected to robot 1 through range factors, and robot 0 is anchored by the prior. This lets the optimizer infer a consistent relative placement for all three robots.

b) Robot 1 acts as the bridge between robots 0 and 2. Its range measurements connect the left and right subgraphs, so information can propagate through robot 1 and tie the whole system together.

c) If robot 1 were removed entirely, then robots 0 and 2 would become disconnected components. Robot 0 would still be anchored by the prior, but robot 2 would only be defined in its own local frame. So the merged map would no longer have a unique global alignment between those two robots.

Q6. Consider two robots that each build their own local map via ICP. You want to merge them into one global coordinate frame using range factors alone.

(a) Is one range measurement enough to determine the relative pose? Why or why not?

(b) What about two range measurements - is there a unique solution?

(Hint: think about what degrees of freedom a single range measurement constrains.) [4 pts]

a) No, one range measurement is not enough to determine the relative pose. In 2D, the relative pose between two robot maps has three degrees of freedom: X, Y, and rotation. A single range measurement only constrains the distance between two poses, so it gives just one scalar constraint. That means many different relative translations and rotations could satisfy the same range.

b) Two range measurements are still generally not enough to give a unique solution. They provide more information, but there can still be multiple relative poses that satisfy both distances, especially because rotation is still not directly constrained. In general, range-only constraints are ambiguous unless you have enough independent measurements from different locations to remove the possible mirrored or rotated solutions.

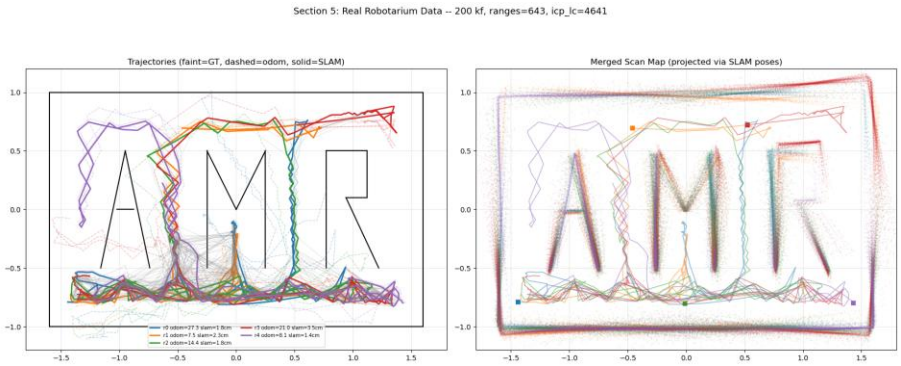
Q7. You submitted two experiments to the Robotarium, each with different randomized exploration parameters. Analyze your results as you would for a real robotics experiment.

(a) For your best run, can you identify the hidden obstacle layout? Describe what you see. Are walls crisp, are there ghosting artifacts, are any regions of the map missing or distorted?

(b) For your worst run, what went wrong? Point to specific artifacts in the map and explain what might have caused them (e.g., poor coverage, too few loop closures, odometry drift in a particular region).

(c) What exploration parameters differed between your two runs (check the experiment log)? Which parameters do you think had the most impact on map quality, and why?

Include screenshots of the scan map and the experiment log showing the randomized parameters for each run. [10 pts]

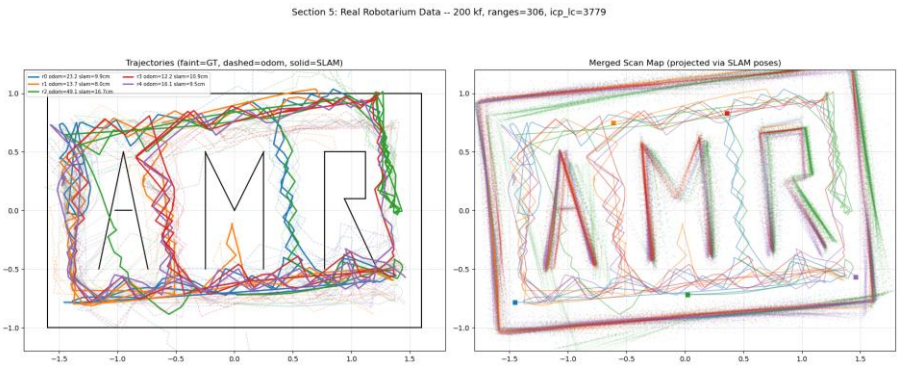


Run 1

Loaded: 5 robots, 200 keyframes, 643 range edges, 6000 timesteps
r0: odom 27.3 cm, slam 1.8 cm
r1: odom 7.5 cm, slam 2.3 cm
r2: odom 14.4 cm, slam 1.8 cm
r3: odom 21.0 cm, slam 3.5 cm
r4: odom 8.1 cm, slam 1.4 cm
Section 5: ranges=643, icp_lc=4641

```
/usr/local/lib/python3.10/dist-packages/matplotlib/projections/__init__.py:63: UserWarning: Unable to import Axes3D.
This may be due to multiple versions of Matplotlib being installed (e.g. as a system package and as a pip package). As a
result, the 3D projection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "
[INFO] [1775160531.218428863] [robotarium]: BARRIER DEADLOCK DURING INITIALIZATION CREATING RANDOM POINT TO ALLEVIATE!
[INFO] [1775160534.897798594] [robotarium]: BARRIER DEADLOCK DURING INITIALIZATION CREATING RANDOM POINT TO ALLEVIATE!
[INFO] [1775160538.598171262] [robotarium]: BARRIER DEADLOCK DURING INITIALIZATION CREATING RANDOM POINT TO ALLEVIATE!
[INFO] [1775160540.309021561] [robotarium]: BARRIER DEADLOCK DURING INITIALIZATION CREATING RANDOM POINT TO ALLEVIATE!
[INFO] [1775160559.289061902] [robotarium]: Starting Experiment
= Experiment Config =
seed = 1386502419
forward_speed = 0.173
turn_omega = 1.731
detect_dist = 0.275
turn_steps_range = (10, 49)
[step    0/6000] keyframes=1, range_edges=0, elapsed=0.0s
[step  500/6000] keyframes=17, range_edges=18, elapsed=16.7s
[step 1000/6000] keyframes=34, range_edges=57, elapsed=33.4s
[step 1500/6000] keyframes=51, range_edges=92, elapsed=50.0s
[step 2000/6000] keyframes=67, range_edges=153, elapsed=66.7s
[step 2500/6000] keyframes=84, range_edges=236, elapsed=83.3s
[step 3000/6000] keyframes=101, range_edges=287, elapsed=100.0s
[step 3500/6000] keyframes=117, range_edges=311, elapsed=116.6s
[step 4000/6000] keyframes=134, range_edges=332, elapsed=133.3s
[step 4500/6000] keyframes=151, range_edges=440, elapsed=149.9s
[step 5000/6000] keyframes=167, range_edges=519, elapsed=166.6s
[step 5500/6000] keyframes=184, range_edges=571, elapsed=183.3s

= EXPERIMENT COMPLETE =
Total steps: 6000
Keyframes: 200
Range edges: 643
Wall-clock time: 199.9s
Saved: gt_poses.npy, encoder_data.npy, lidar_scans.npy, keyframe_steps.npy, range_edges.npy, initial_poses.npy
```



Run 2

Loaded: 5 robots, 200 keyframes, 306 range edges, 6000 timesteps
r0: odom 23.2 cm, slam 9.9 cm
r1: odom 13.7 cm, slam 8.0 cm
r2: odom 49.1 cm, slam 16.7 cm
r3: odom 12.2 cm, slam 10.9 cm
r4: odom 16.1 cm, slam 9.5 cm
Section 5: ranges=306, icp_lc=3779

```
[INFO] [1775072191.392665898] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.397938326] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.403658480] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.408868821] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.414271902] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.419906488] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.425748569] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072191.431109510] [robotarium]: Exception occurred: Starting the Experiment Failed... The middleware was
unable to start the experiment
[INFO] [1775072219.303532003] [robotarium]: Starting Experiment
= Experiment Config =
seed = 999053884
forward_speed = 0.167
turn_omega = 1.883
detect_dist = 0.240
turn_steps_range = (11, 40)
[step    0/6000] keyframes=1, range_edges=0, elapsed=0.0s
[step  500/6000] keyframes=17, range_edges=29, elapsed=16.7s
[step 1000/6000] keyframes=34, range_edges=72, elapsed=33.4s
[step 1500/6000] keyframes=51, range_edges=87, elapsed=50.0s
[step 2000/6000] keyframes=67, range_edges=104, elapsed=66.7s
[step 2500/6000] keyframes=84, range_edges=117, elapsed=83.4s
[step 3000/6000] keyframes=101, range_edges=139, elapsed=100.0s
[step 3500/6000] keyframes=117, range_edges=168, elapsed=116.7s
[step 4000/6000] keyframes=134, range_edges=215, elapsed=133.9s
[step 4500/6000] keyframes=151, range_edges=231, elapsed=150.6s
[step 5000/6000] keyframes=167, range_edges=246, elapsed=167.3s
[step 5500/6000] keyframes=184, range_edges=259, elapsed=183.9s

= EXPERIMENT COMPLETE =
Total steps: 6000
Keyframes: 200
Range edges: 306
Wall-clock time: 200.6s
Saved: gt_poses.npy, encoder_data.npy, lidar_scans.npy, keyframe_steps.npy, range_edges.npy, initial_poses.npy
```

a) It is honestly a little hard to say which run looks better just from the map. Both runs clearly recover the A, M, and R wall layout. Run 2 looks a bit sharper in some local parts of the letters, but it is also more tilted and distorted overall. Run 1 looks a little less sharp in places, but the global structure is more consistent.

b) The numbers favor run 1. Even though run 2 can look better locally, run 1 is more accurate overall and stays much better aligned globally. Run 2 seems to suffer from more drift, especially in the overall orientation and boundary shape.

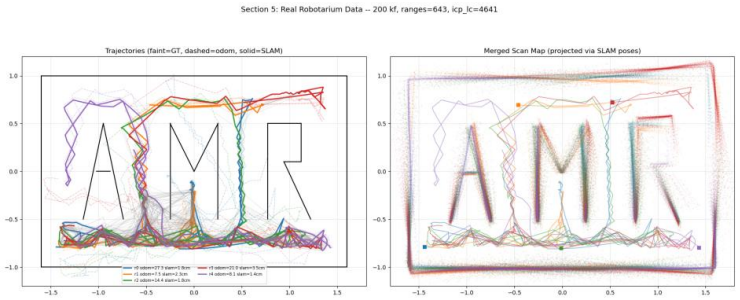
c) The biggest difference was probably the exploration pattern. The run that gave better coverage and more useful inter-robot connections ended up producing the better global map. The other run still found the wall shapes, but the weaker global constraints seem to have let more drift accumulate, which is why it ended up tilted even though some local features looked sharper.

Q8. Compare your best Robotarium scan map (Section 5) with your Section 4 simulation map.

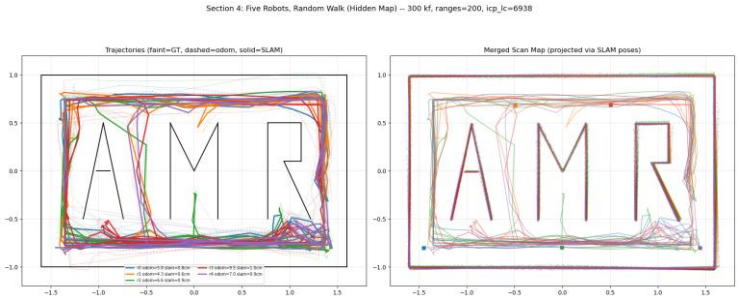
(a) Are there visible differences in map quality?

(b) What might cause them? Consider: real encoder noise vs simulated, wheel slip, placement imprecision, lidar simulation fidelity, etc.

Include screenshots of both maps side by side. [7 pts]



Section 5



Section 4

a) Yes, there are visible differences, but they are not huge. The Section 4 simulation map looks cleaner overall, with straighter boundaries and less spread in the wall points. The best Section 5 Robotarium map still recovers the A, M, and R layout well, but it has a bit more thickness, ghosting, and small distortions. Run 2 on the robotarium was able to get better results for the inner walls though.

b) The main reason is that the real run includes effects that are harder to model perfectly in simulation. Real encoder measurements are noisier and their actual placement and motion are not perfectly repeatable. Small errors in wheel motion or starting position can accumulate over time and make the global map less crisp.

Q9. What was the most surprising or interesting thing you observed during this project?
Open-ended. Anything about ICP, factor graphs, map quality, real vs simulated, etc.
If applicable, include a screenshot that illustrates what surprised you. [4 pts]

The most surprising thing was that a map can look sharper locally but still be worse overall. In one Robotarium run, the A, M, and R walls looked a bit crisper, but the whole map was more tilted and had larger errors. So even though visually it might seem like a better map, quantitatively the numbers were a lot worse.

Q10. Early Bird Robotarium (extra credit)

Paste a screenshot of *your* Robotarium dashboard that shows an experiment that has been *submitted* before Friday, March 20, 5 PM ET. The entire page (Robotarium logo + Pending Experiments + Completed Experiments) should be visible - you may need to zoom out to make it fit. If you did not submit early, leave this blank.